

EduLearn

Production readiness

Audit, data integrity, deletion safety, timezone correctness and feature completion

Generated 2026-10-05

Scope Laravel 13 · PHP 8.3 · MySQL · server-rendered Blade

Verification 257 automated tests passing against an isolated MySQL database

Distribution Generated locally. Not published or uploaded.

EduLearn — architecture and operating reference

Written after the production-readiness work described below. It records what the system does today, not what is planned. Diagrams live in `docs/diagrams/`; the requirement-by-requirement status stays in `docs/requirements-status.md`.

1. Existing architecture

A server-rendered Laravel 13 application on PHP 8.3 with MySQL. Blade renders every page. There is no npm manifest, no bundler and no frontend framework: the interface is one hand-written stylesheet and one progressive-enhancement script, both served from `public/`. A strict Content-Security-Policy (`script-src 'self'`, `object-src 'none'`, `frame-ancestors 'none'`) means there are no inline scripts, styles or event handlers anywhere in the views.

Layering is deliberately shallow:

- **Routes** — a single `routes/web.php`, one guest group and one authenticated group.
- **Controllers** — thin. They authorize, delegate, and choose a response.
- **Actions** — every multi-step mutation. This is where transactions, the shared write lock,

re-authorization and optimistic-concurrency checks live.

- **Services** — single-purpose collaborators with no knowledge of HTTP.
- **Models** — Eloquent where a record has behaviour; several tables (quizzes, attempts, audit

trails) are driven through the query builder and have no model by design.

2. Improved architecture

What this work added, and why:

Added	Reason
Actions\WriteLock	The lock was inlined in 19 places and failed open when its row was missing
Services\DisplayTime	One boundary between stored UTC and displayed local time
Services\DependencyAnalyzer	Deletion safety decided from the real foreign-key graph
Actions\ContentLifecycle	Archival, restoration and guarded permanent deletion
Services\RichText	The only route by which authored HTML enters the system
Services\QuizQuestionProvider	AI-drafted questions over the existing provider configuration
Actions\AssignmentMediaLibrary	Reference media validated by content, not by filename
Console\EraseApplicationData	A data reset that keeps the schema and the migration history

3. Database relationships

See `docs/diagrams/erd.md`, generated from the live schema. The shape in one sentence: a course owns its content, a learner's relationship to a course is an enrolment, and everything a learner produces hangs off one of those two.

4. Deletion strategy

Four independent defences, in order:

1. **Authorization** — re-checked against freshly loaded state inside the transaction, never

trusting the state that was read when the page was rendered. 2. **Serialization** — the shared write lock, so two staff cannot race the same decision. 3. **Dependency analysis** — `DependencyAnalyzer` reads foreign keys from `information_schema` and counts referencing rows. It is not a hand-maintained list, so a table added by a future migration is accounted for the day it appears. 4. **Database constraints** — `RESTRICT` refuses the delete even if everything above were wrong.

Constraints are never disabled to make a delete succeed.

5. Soft-delete strategy

There is no `deleted_at` column and no `SoftDeletes` trait. Removal is expressed as a `status` on the record — the idiom courses, materials and video lectures already used, now extended to lessons. Two parallel notions of "removed" would force every query, policy and relation to decide which one it meant.

An archived lesson leaves the learner's view, stops counting toward course progress and refuses completion, while every `lesson_completions` row already earned is retained.

6. Hard-delete safety rules

Permanent deletion is offered only where the analyser reports no blocking reference, and only from Trash, where the volume is small enough to compute a report per record.

- A lesson is blocked by completions, study notes or materials.
- A course is blocked by lessons, enrolments, assessments, announcements, notes or lectures.
- When blocked, the record is **archived instead** and the response names what is being

preserved — "...because 3 learner progress records depend on this record" — rather than showing a constraint violation.

- Course deletion additionally requires an administrator.

7. Authorization model

`CoursePolicy` carries the whole model:

Ability	Granted to
<code>manage</code>	Active admins; active instructors who own the course or co-teach it
<code>view</code>	Anyone who can <code>manage</code> , plus enrolled learners
<code>participate</code>	Active learners enrolled in a published course
<code>enroll</code>	Active learners, published course, not previously removed by an administrator
<code>studySource</code>	Same as <code>participate</code>

Publication is a **visibility** state and never withdraws authoring rights. Course status appears nowhere in `manage`.

8. Quiz lifecycle

See `docs/diagrams/quiz-lifecycle.md`. The rules worth stating in prose:

- The availability window is optional at both ends. Absence is a null column, never a sentinel

date, so "no deadline" cannot become a deadline when some far-future date arrives.

- Publishing a quiz freezes its question set, so an attempt is always scored against the paper it was sat under. Title, instructions, window and duration stay editable.
- An attempt's `deadline_at` is fixed when it starts; moving the window never changes time a learner was already promised.
- Results are released only after the quiz closes — or, when nothing closes, once the learner's own attempt is finalized.

9. Timezone handling

Storage is UTC everywhere; `config('app.timezone')` is UTC and every column holds UTC. Interpretation happens at two edges only, both in `DisplayTime`:

- **Input** — a value typed into a form is read in `lms.display_timezone` (default `Asia/Karachi`) and converted to UTC **before validation runs**. This ordering is the whole point: `after:now` and `after:opens_at` resolve in the application timezone, so a local wall-clock string compared against a UTC instant would reject any window opening within the next five hours.
- **Display** — `@showtime`, `@showdate`, `@inputtime` and `@tz` convert back and name the zone.

No fixed offset is ever added or subtracted; conversion goes through the timezone database.

10. Announcement scheduling

An announcement is a draft (`published_at` null), scheduled (future) or published (past). Visibility is a **query condition**, not a background job: it becomes visible because its time has passed, so a stopped worker can neither hold one back nor release one early. Unpublished announcements can be edited or discarded by their author; published ones cannot — a correction is published instead of silently rewriting what people already read.

11. Media upload workflow

Three separate upload paths, each validated by **content** rather than by the filename or the browser-supplied MIME type:

Path	Accepted	Validated by	Ceiling
Study materials	PDF, DOCX, PPTX, TXT, MD	<code>Rules\StudyMaterialFile</code> (zip entry audit, OOXML namespace, no macros)	10 MB
Video lectures	MP4 / H.264 / AAC	<code>Services\VideoProbe</code> (ISO-BMFF box tree, in PHP)	<code>video.max_kilobytes</code> , 500 MB
Assignment reference media	JPEG, PNG, WebP, MP4	<code>getimagesize</code> / <code>VideoProbe</code>	<code>lms.assignment_media.*</code>

The three ceilings are independent on purpose: raising one must never raise another. Stored names are 32-hex generated, so a crafted client filename cannot influence a path. Nothing is served from a public URL; delivery goes through an authorized route, and video through `PrivateMediaStream` so Range requests work without buffering the file. No subprocess is ever spawned and no uploaded filename ever reaches a shell.

12. AI quiz workflow

`QuizQuestionProvider` reuses the provider configuration, encrypted credential, zero-retention settings and failure classification that already serve study notes.

- Generated questions are appended to a **draft** quiz as ordinary draft questions. A published quiz refuses them outright.
- Malformed, duplicated or unanswerable items are discarded rather than guessed at: four distinct options and an in-range answer, or the question is dropped.
- Provider failures are translated into guidance for an administrator. Response bodies are never surfaced, because they can carry account detail.
- Course material is framed to the model as untrusted data, never as instructions.
- Nothing is published automatically. A person reviews and publishes.

13. Error-handling strategy

User-facing messages are written for the person; technical detail stays server-side.

- One notification mechanism: flash messages render server-side into a toast region with two live regions split by urgency. They are readable before any script runs and survive its absence; the script only adds dismissal and timing.
- Standing guidance stays as inline `.notice` content, because it belongs in the page rather than in something that disappears.
- Error views render only application-authored `abort()` messages, never exception internals.
- Deletion refusals explain what is being preserved instead of surfacing a constraint violation.

14. Empty database behaviour

An installation holding only login accounts is a supported state. `EmptyDatabaseTest` exercises 13 screens across all three roles plus login, logout and an empty search. No crash was found, so no null-safety churn was introduced: the layout's site-name lookup already falls back to config, every list uses `@forelse`, and `AiSettings::current()` falls back when no row exists.

Two rows are genuinely required and are recreated by both the erase command and the minimal seed: `lms_write_locks` row 1, and the `site_name` setting.

15. Seeding strategy

Command	Purpose
<code>php artisan lms:erase</code>	Delete application data, keep schema and migration history
<code>php artisan db:seed --class=MinimalLoginSeeder</code>	Three role accounts, no content
<code>php artisan db:seed</code>	Complete development dataset

The complete seed deliberately covers states the happy path never reaches: draft, archived and content-free courses; an archived course that retains its enrolment; an archived lesson; a graded and published submission; a finalized quiz attempt; a quiz with no window; and live, scheduled and draft announcements.

16. Testing strategy

PHPUnit against an isolated MySQL database whose name must end in `_test`; `tests/TestCase.php` refuses anything else before `RefreshDatabase` can drop a table. There is no SQLite run.

Tests assert behaviour at the HTTP boundary wherever possible, because the property that matters is "this cannot be stored" rather than "this function returns a clean string". Security properties are asserted negatively — a disguised executable, a `javascript:` URL, a stale form, a learner reaching another course — so a regression fails loudly.

17. Major improvements made

Foundations: a fail-loud shared write lock; remember-me fixed; scheduled announcements no longer publish immediately; documentation drift corrected.

Correctness: published-course content is editable with integrity rules on what submissions make immutable; a timezone layer with conversion ordered before validation; optional quiz windows.

Integrity: Trash and restore over the archive idiom, with permanent deletion gated by the real foreign-key graph; a data-only erase command.

Content and UX: a sanitized rich-text editor; assignment reference media; one toast mechanism; optional reason fields where they are not audit evidence.

AI: draft quiz generation over the existing provider abstraction, review required before publication.

18. Remaining technical debt

- **Not verified**, as distinct from not built: real SMTP delivery, live external AI inference,

WCAG 2.1 AA conformance, and load or availability targets. No browser or assistive technology was driven during this work.

- `Course::progressFor()` still issues two queries per call and is called once per course card.
- Several screens run model queries directly in Blade, and the layout reads `settings` on every

render.

- A 409 from a stale form still renders a full-page error view, which loses the form input.
- Assignment reference media cascades from `assignments`; assignments have no delete path today,

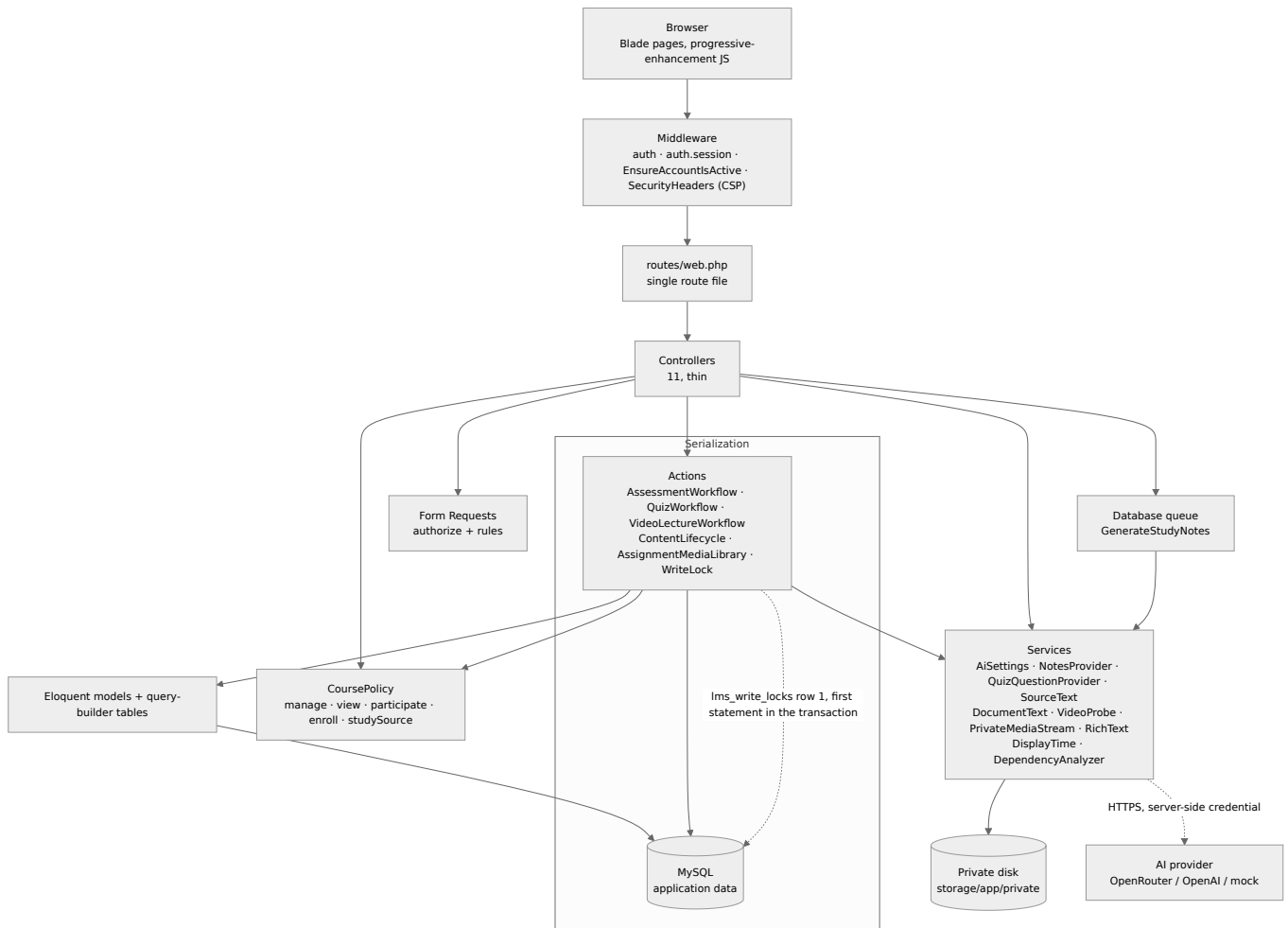
so this is latent rather than active.

- `league/commonmark` was updated to 2.10.3 during this work to clear two advisories; dependency auditing is not otherwise automated.

- Quiz attempt limits and a configurable grading or rounding policy remain unbuilt, as recorded in `docs/requirements-status.md`.

A. System architecture

Request path through the application. Blade renders every page; there is no client-side router, no API consumed by a frontend, and no build step.

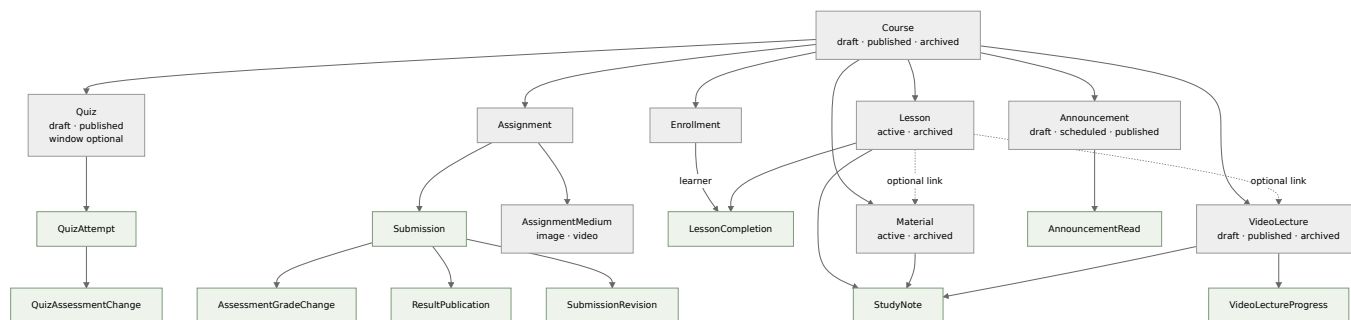


Notes that are easy to miss:

- Every multi-step mutation takes the shared write lock before reading anything, then re-checks authorization against freshly loaded state inside the same transaction.
- Private files never have a public URL. Bytes leave only through an authorized route, and video goes through `PrivateMediaStream` so Range requests work without buffering the file.
- The AI provider is reached only from the server, with the credential encrypted at rest.

B. Course and learning flow

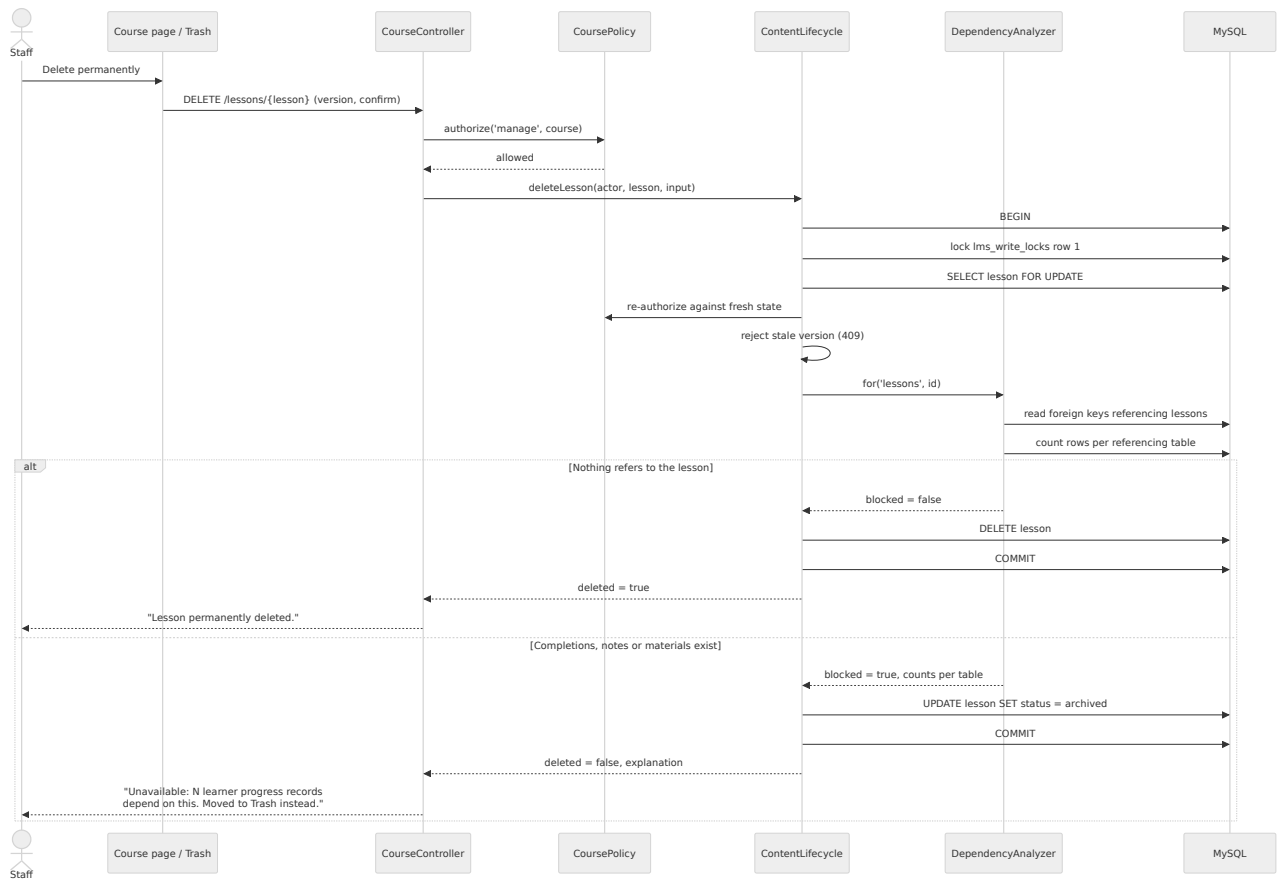
Derived from the actual schema; every arrow is a real foreign key.



Shaded nodes are learner history. Each one is a `RESTRICT` reference, which is what makes the parent undeletable while it exists.

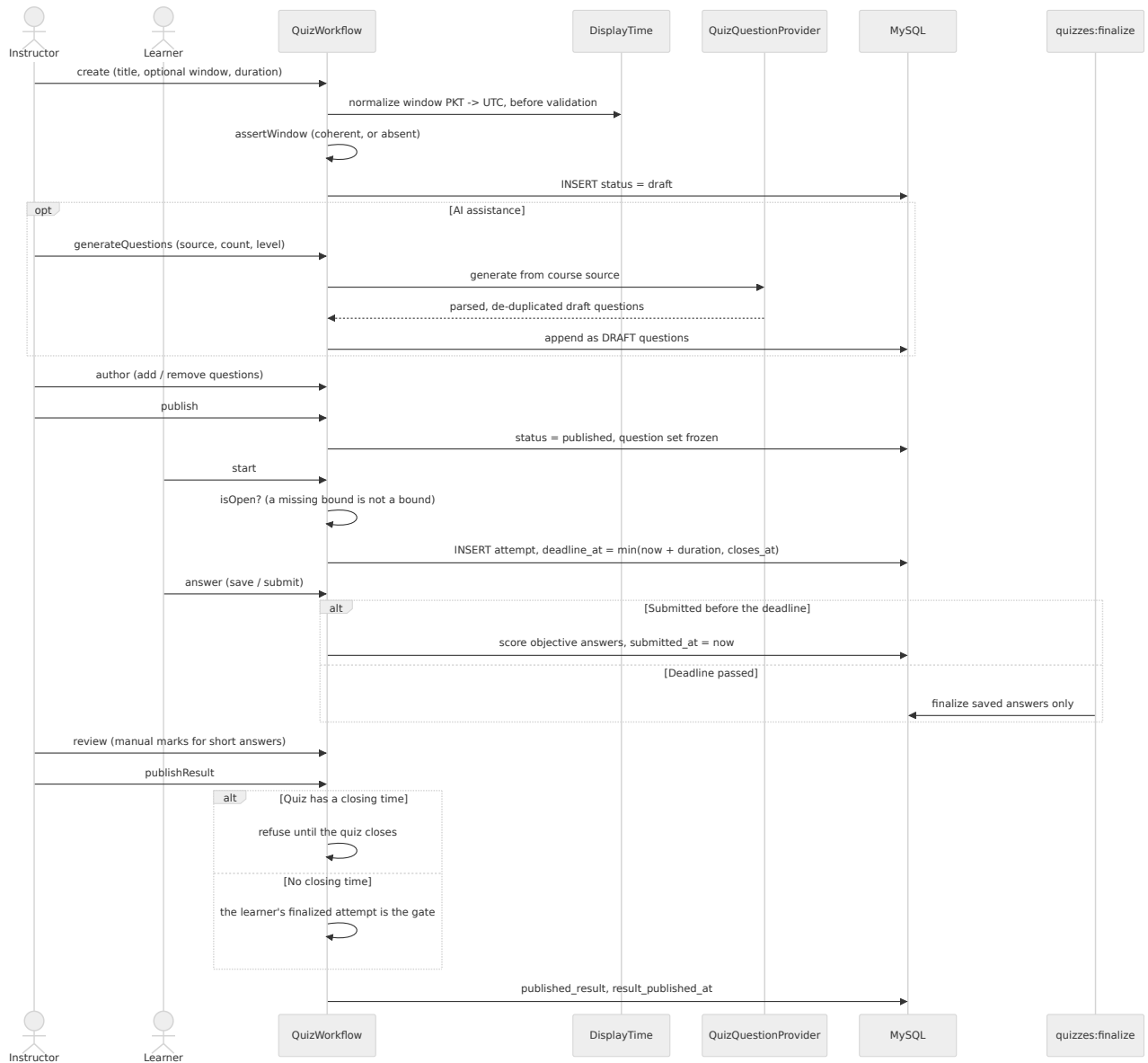
C. Lesson deletion

The implemented path, not an idealised one.



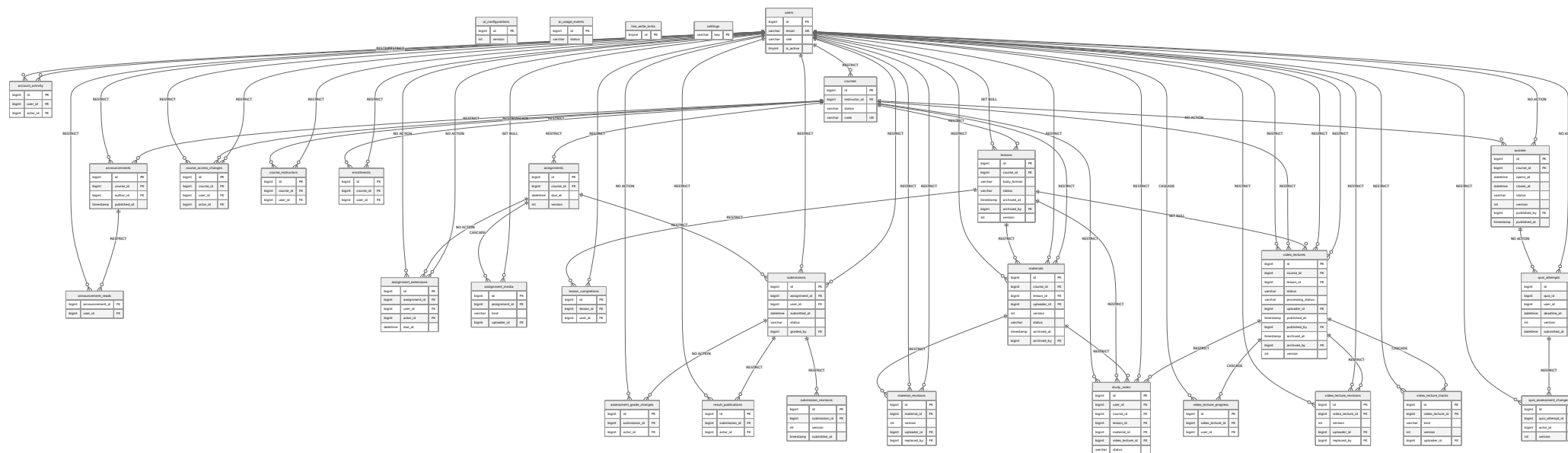
The database constraints remain the final guard: if the analyser were ever wrong, **RESTRICT** refuses the delete rather than orphaning a record. Constraints are never disabled.

D. Quiz lifecycle



Editing after publication changes the title, instructions and window only. Attempts keep the `deadline_at` recorded when they began, so moving the window never alters time already promised.

Keys and state-carrying columns only; the full column list is the schema itself. Each relationship is labelled with the database's own `ON DELETE` action, because that action is what ultimately decides whether a record can be destroyed.



Reading the delete rules

Action	Meaning	Where it is used
RESTRICT / NO ACTION	The parent cannot be deleted while the child exists	All learner history and all authored content
SET NULL	The child survives and forgets the parent	video_lectures.lesson_id , lessons.archived_by , uploader columns
CASCADE	The child is destroyed with the parent	Only video_lecture_progress , video_lecture_tracks and assignment_media , none of which is assessed work

Dangerous relationships, stated plainly. `assignment_media` cascades from `assignments`: if an assignment were ever deletable, its reference files would go with it. That is correct for an illustration and would be wrong for a submission, which is why `submissions` restricts instead. `video_lecture_progress` cascades from both the lecture and the user — viewing position is a convenience, not a grade, and the design treats it as such. Nothing that records an assessment outcome cascades anywhere.

EduLearn requirements reconciliation

Baseline re-verified against code, schema and a full MySQL test run on **24 September 2026**. Status describes source implementation and automated-test evidence, not production acceptance. Runtime evidence is in `validation.md`; video specifics are in `video-operations.md`.

Suite at the time of writing: **257 tests, all passing** on MySQL (`acumen_university_test`). This supersedes the earlier 89-test and 110-test baselines; see `docs/architecture.md` for the work that followed.

Architecture and compatibility

The repository is a Laravel 13.31.0 modular application running PHP 8.3.31, using MySQL for persistent application data, Eloquent, Blade, private local storage, database sessions/cache/queues, and PHPUnit 12.5.34. The UI uses existing local CSS and small progressive-enhancement JavaScript. There is no npm package manifest or frontend framework build.

Laravel 13 requires PHP 8.3 or newer; the SRS's PHP 8.2 minimum is incompatible. Keep the installed Laravel/PHP major versions instead of introducing an unrelated upgrade. See [Laravel 13 support policy](#), [deployment requirements](#), and [PHP support windows](#). The running MySQL version must be verified during environment validation. SQLite is no longer a supported test backend: the harness requires an isolated MySQL database whose name ends in `_test`.

The specification both permits retaining the current frontend and requests Bootstrap. Decision: retain the established stack and visual components for milestone one. No Bootstrap, jQuery, Alpine or npm dependency is claimed to be installed. A future build-tool change must have a concrete reason. EduLearn replaces the Acumen display name, while retaining existing account identifiers and the restrained green visual identity.

Requirements checklist

Requirement	Status	Evidence or remaining work
Three primary roles and server authorization	Implemented	Admin/instructor/student storage roles; student is displayed as learner. Ownership/co-instructor/enrollment policy applies to course operations
Login/logout and role dashboard redirects	Implemented	LoginRequest, AuthController, role dashboard routes, session regeneration/invalidation
Five failed-attempt logout	Implemented	Hashed normalized email key, five failures in a configurable window; separate lock expires 15 minutes after the fifth failure by default; IP request throttling is additional
Password reset	Partial	Laravel broker and automated notification tests exist; real SMTP delivery remains to be configured/verified
Administrative account provisioning/update	Implemented	Admin account forms, role validation, duplicate-email validation, account revision checks and safe activity records
Deactivation and existing-session revocation	Implemented	Active-account middleware, auth version checks, remember-token rotation, database-session removal; reactivation does not restore old sessions
Profile/password updates	Implemented	Current-password validation, restricted fields, unique email, session revocation after password change
Public signup	Out of scope	Accounts are administrator-provisioned
Learner self-enrollment	Implemented	Latest user request overrides the initial admin-only policy; published courses only, duplicate prevention, audited enrollment and administrator revocation protection
Course title/description/status/instructor	Implemented	Existing management screens, primary and additional instructors, archive retains learning records
Unique course code	Implemented	Unique database constraint, normalized form validation, preserved-record backfill, code search and course display
Administrator-managed course enrollment	Implemented	Audited access screen and unique online enrollment constraint
PDF/DOCX/PPTX materials, 10 MB, complete metadata	Implemented and tested	10 MB private uploads, Office content validation, uploader/size/time metadata, progress feedback. Replacement with optimistic version checks, retained file revisions, staff-only revision downloads, archival that withdraws learner access and blocks new AI generation while preserving existing notes. <code>MaterialController</code> , <code>2026_09_15_070733_add_material_revision_history</code> , <code>MaterialLifecycleTest</code> (4 tests). Historical unknown uploader/size legitimately remain null
Assignment authoring and private submissions	Implemented	Title/instructions/deadline/marks, text/file submissions, private downloads, post-publication editing with <code>max_marks</code> frozen once work is submitted, and reference image/video attachments. <code>PublishedCourseEditingTest</code> , <code>AssignmentMediaTest</code> . Assignment draft/publish states remain
Reject late submissions	Implemented	Server rejects at or after the effective UTC deadline; individual approved extensions apply
Submission history and replacement policy	Implemented	Ungraded work may be replaced before deadline; retained text/private file revisions, authorized history downloads, duplicate-content idempotency and form version guard

Requirement	Status	Evidence or remaining work
Rubrics, marks, feedback and grade corrections	Implemented	Bounded decimal marks, criterion totals, immutable rubric after first submission, grade history and stale-form protection
Result draft/publication states	Implemented	<code>AssessmentWorkflow::publishResult</code> , <code>result_publications</code> , published snapshots, corrections stay draft until republished. <code>ResultPublicationTest</code>
Configurable grading/rounding policy	Missing	No course grading-policy administration yet
Timed MCQ quizzes	Implemented	Draft review, immutable publication, one resumable attempt, saved answers, server deadline and objective scoring
Short-answer quizzes	Implemented	Authoring, manual bounded decimal grading, review audit and explicit publication. <code>ShortAnswerQuizTest</code> , <code>QuizWorkflow::review</code>
Visible countdown/configurable attempt limit	Partial	Visible countdown and automatic server finalization request added; server remains authoritative. Configurable attempt limits remain
Expired quiz finalization	Partial	Idempotent scheduled command exists; host scheduler supervision must be configured and verified
Course announcements	Implemented	Scoped course announcements with author and publication metadata, unread tracking, and draft/scheduled/published states decided at read time. <code>AnnouncementAudienceTest</code> , <code>ScheduledAnnouncementTest</code>
Platform announcements	Implemented	Administrator-published platform notices with scheduling. Assignment/submission notifications remain missing
Private AI study notes	Partial	Queue, private library, limits, retries, source hash/version checks, mock/direct OpenAI provider paths exist
Document extraction	Implemented	Lesson text, TXT, Markdown, DOCX, PPTX and text-based PDF. Scanned PDFs are reported as needing OCR, which is not implemented. <code>DocumentExtractionTest</code>
Note editing/regeneration and provider/model metadata	Partial	Rename/delete exists; content editing, explicit regeneration and complete model metadata remain
Instructor AI question generation/review	Implemented	<code>QuizQuestionProvider</code> drafts questions from a course source into a draft quiz; malformed, duplicate and unanswerable items are discarded, a published quiz refuses them, and nothing is published without review. <code>AiQuizGenerationTest</code>
OpenRouter/free models/AI administration	Implemented	Live per-provider catalogues, encrypted credentials, zero-price enforcement with no paid fallback, and connection checks. <code>AiAdministrationTest</code>
Learner dashboard	Partial	Enrolled courses/progress, deadlines, quiz windows, materials, course announcements and notes; published-result feed/general announcements await later milestones
Instructor dashboard	Partial	Assigned courses, enrollments, pending reviews and assessment windows; richer submission history awaits milestone 3
Admin dashboard/activity	Partial	Database-backed course/enrollment/account counts, management links and recent account activity; fuller reports await milestone 7
Responsive/accessible shared UI	Partial	Focus styles, labels, skip link, pagination, responsive layout and choice controls; WCAG 2.1 AA not certified

Requirement	Status	Evidence or remaining work
Performance/availability targets	Missing	No representative 100/200-user load run or availability observation period completed
Campus ERP/attendance/degrees/physical timetables	Out of scope	Application modules removed; retained historical schema fixtures are upgrade-test inputs only
Payments/certificates/video/forums/mobile/advanced analytics	Out of scope	Separate future backlog, not core milestone deliverables

Production-readiness round (October 2026)

Added since the milestone baseline above, each verified by automated tests: a fail-loud shared write lock; remember-me restored; post-publication editing of lessons, assignments and quizzes with integrity rules; an app-wide display timezone with conversion ordered before validation; optional quiz availability windows; Trash, restore and dependency-gated permanent deletion; a data-only erase command with minimal and complete seeds; one toast notification mechanism; optional reason fields where they are not audit evidence; sanitized rich-text lessons; assignment reference media; and AI-drafted quiz questions requiring review.

Full detail is in `docs/architecture.md`; diagrams are in `docs/diagrams/`.

Assumptions and contradictions

- Accounts are administrator-managed. The latest user request enables learner self-enrollment in published courses alongside administrator enrollment. Public registration remains disabled.
- Store timestamps in UTC and label UTC displays. Localized display and timezone-aware maintenance preferences require later work.
- Assignment replacement will be allowed only before the effective deadline, keeping immutable versions. Strict rejection at and after the effective deadline is implemented.
- Quiz initial policy is one attempt. Later configuration must not silently grant more attempts or change an already-started attempt's deadline.
- Result release must be explicit. Existing immediately visible grades are legacy behavior that milestone 4 replaces.
- The 10 MB/10 second upload target at 5 Mbps is infeasible: 10 MB takes roughly 16 seconds before overhead (10 MiB roughly 16.8 seconds). Measure network transfer and server processing separately; proposed acceptance is transfer consistent with available bandwidth plus a separately measured server-processing budget, to be agreed before performance acceptance.
- Proposed peak-load acceptance: at 200 concurrent users, no lost/duplicate assessment writes and p95 responses below five seconds; normal-load p95 below three seconds. These are proposed measurable thresholds, not achieved results.
- Availability target: 99% per calendar month, excluding only preannounced approved maintenance. Maintenance timezone, weekday 02:00–04:00 window and 24-hour notice remain configurable requirements, not implemented scheduling.
- Production deployment and destructive changes require explicit authorization. The prior online-scope request authorized campus removal; its conversion has a separate backed-up retention plan. No Git push/pull/fetch or deployment is authorized.

Video lectures (new core requirement)

Requirement	Status	Evidence
Authoring by administrators and assigned instructors	Implemented and tested	<code>VideoLectureController</code> , <code>VideoLectureWorkflow</code> , course manage gate
Course/lesson association, title, description, display order	Implemented and tested	<code>video_lectures</code> schema, <code>updateDetails</code>
Draft / published / archived states with timestamps and actors	Implemented and tested	<code>changeStatus</code> ; publication blocked unless media is <code>ready</code>
Upload metadata: uploader, filename, MIME, container, size, duration, processing status	Implemented and tested	Recorded from <code>VideoProbe</code> at upload
Safe replacement with version retention	Implemented and tested	<code>video_lecture_revisions</code> , optimistic <code>version</code> check, staff-only revision downloads
Optional poster/thumbnail	Implemented	Uploaded and content-verified image; not generated from a frame (needs FFmpeg)
Captions and transcript management	Implemented and tested	Validated WebVTT, plain-text transcript, <code>video_lecture_tracks</code>
Documented supported formats	Implemented	MP4 / H.264 / AAC only — <code>docs/video-operations.md</code>
Media validated with an appropriate tool	Implemented and tested	<code>VideoProbe</code> parses the ISO-BMFF box tree in pure PHP. ffprobe is not installed ; no subprocess is spawned and no filename reaches a shell
Transcoding	Not implemented	Documented. <code>processing_status</code> exists so a queued stage can be added without migration
Separate configurable video limit, document limit unchanged	Implemented and tested	<code>config/video.php</code> (500 MB default); the 10 MB material limit is untouched and separately asserted
Aligned app/PHP/web-server limits	Blocked on external configuration	Written into <code>~/.config/valet/Nginx/lms.test</code> ; nginx has not been reloaded (needs sudo), so the live site still enforces 10M
Resumable/chunked upload	Not implemented	Single request with progress, cancellation and safe retry. Not claimed as supported
Private storage, generated filenames, no public URLs	Implemented and tested	Private disk, 32-hex names, authorized routes only
Server-authorized playback incl. deactivation and revocation	Implemented and tested	Active account + published course + enrollment + published/ready lecture
HTTP Range / HEAD, 206 and 416	Implemented and tested	<code>PrivateMediaStream</code> ; verified in tests and live over HTTPS on the Valet site
No whole-file buffering	Implemented	256 KB chunked streaming from a file handle
Optional internal nginx delivery	Implemented	<code>VIDEO_X_ACCEL_PREFIX</code> , off by default
Responsive, keyboard-accessible player with speed and captions	Implemented	Native controls + explicit speed select, <code>aria-live</code> states, 16:9 frame, no autoplay. Not yet verified with assistive technology
Playback position saved and resumed	Implemented and tested	<code>video_lecture_progress</code>
Completion excludes seeking; no double counting	Implemented and tested	Credit = min(client delta, new forward ground, wall-clock elapsed x 2.5), first report additionally capped. A live check found a short-lecture hole, which was fixed and regression-tested
Viewing progress distinct from mastery	Implemented	Kept separate from lesson-completion course progress
Learner records private; staff see their course	Implemented and tested	Progress roster on the lecture page
Transcript as an AI note source	Implemented and tested	<code>source_type=lecture</code> , reuses quota/queue/authorization. Automatic transcription is not claimed

Requirement	Status	Evidence
External embeds	Out of scope	Uploaded private playback is the delivered core; no provider allowlist was added
DRM	Out of scope, explicitly	Access is controlled; redistribution of delivered bytes is not preventable

Document text extraction

Source	Status
Lesson text, UTF-8 TXT, Markdown	Implemented and tested
DOCX	Implemented and tested — WordprocessingML runs, paragraph order preserved
PPTX	Implemented and tested — DrawingML runs, slide order preserved
PDF (text-based)	Implemented and tested — raw and Flate streams, <code>Tj</code> / <code>TJ</code> / <code>'</code> / <code>"</code> operators, escapes
Scanned/image-only PDF	Reported as needing OCR. No OCR is implemented
Encrypted PDF	Detected and reported
Empty/malformed documents, oversized extraction	Detected and reported

Evidence: `App\Services\DocumentText` , `DocumentExtractionTest` (9 tests).

Milestone plan

- 1. Accounts and dashboard foundation:** provision/update/deactivate, logout, profiles/passwords, role redirects, access revocation, useful database-backed dashboard sections. Implemented in source; see validation results.
- 2. Courses and materials:** unique codes, assignment control, administrator and learner enrollment, 10 MB PDF/DOCX/PPTX validation and metadata, safe material lifecycle.
- 3. Assignments and communication:** publication/attachments, hard deadlines, retained submission versions, feedback, dashboard notifications, platform/course announcement audiences.
- 4. Quizzes and released results:** short answers/manual review, countdown/recovery, configurable attempts, grading policy and draft/published results.
- 5. AI study notes:** supported document extraction, structured source-grounded notes, edit/regenerate, full provider/model metadata and failure handling.
- 6. Instructor AI and administration:** draft question review/acceptance, OpenRouter model discovery, encrypted credentials, quotas, connection testing, failure/spend visibility.
- 7. Acceptance verification:** reports, responsive/keyboard/accessibility review, security regression, representative MySQL load tests, queue/scheduler supervision and operating runbooks.

Current execution gate

Milestone one is implemented and tested on isolated databases. Following explicit user approval, the scope conversion and account migrations were applied to the local MySQL database. Valet browser smoke checks passed at mobile, tablet and desktop widths. Real email delivery and full accessibility acceptance remain unverified.

Still outstanding (as of 24 September 2026)

Not started or incomplete, stated plainly so the checklist is not read as completion:

Item	Status
Assignment draft/published workflow and instructor attachments	Missing
Assignment/submission dashboard notifications	Missing
Configurable quiz attempt limits	Missing — policy remains exactly one attempt
Configurable grading and rounding policy	Missing
Published-result dashboard summaries and role-scoped reports	Missing
Instructor AI quiz-question generation	Missing
Queue/scheduler supervision on a real host (systemd/Supervisor)	Not configured
SMTP delivery verification	Not verified — mailer is <code>log</code> locally
nginx reload to apply the video upload limits	Blocked — needs sudo
Accessibility conformance (WCAG 2.1 AA)	Not verified — no assistive-technology or visual responsive testing was performed
Performance and availability targets	Not measured — no load test was run
Real external AI inference	Not proven — provider paths are covered only by mocked HTTP and an auth-only connection check